BOTSWANA INTERNATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY

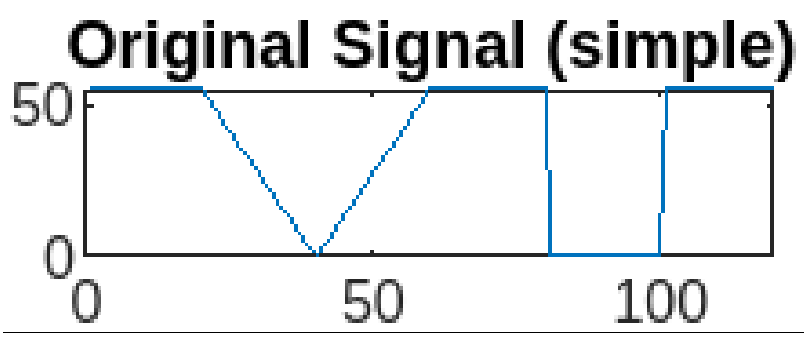
NAME	SURNAME	STUDENT ID	TIME SPENT
ISHEANESU	**MUTAMBA**	**20000721**	**12hrs**
Simangaliso	**Mdluli**	**21000300**	**12hrs**

Part 1.

```
% Load the data
load('lab2_data.mat'); % Contains 'simple' (noise-free) and 'simple_noise' (with
random noise)

%% Part 1a: Examining Delay Effects
% Define a 7-point running average filter
bb = ones(7, 1) / 7;

% Apply the filter to 'simple' using the filter command
yy1 = filter(bb, 1, simple);

% Apply the filter to 'simple' using the filter2 command (with 'same' parameter)
yy2 = filter2(bb, simple, 'same');

% Plot the original and filtered signals
figure;
subplot(3,1,1);
plot(simple);
title('Original Signal (simple)');

subplot(3,1,2);
plot(yy1);
title('Filtered Signal using filter');

subplot(3,1,3);
plot(yy2);
title('Filtered Signal using filter2');

% Calculate the Mean-Squared Errors (MSE)
```

```
mse_filter = mean((simple - yy1).^2);
mse_filter2 = mean((simple - yy2).^2);

disp(['MSE using filter: ', num2str(mse_filter)]);
disp(['MSE using filter2: ', num2str(mse_filter2)]);


%% Part 1b: Measuring Distortion in 1-D
% Apply the same 7-point running average filter to the noisy signal 'simple_noise'
yy3 = filter2(bb, simple_noise, 'same');

% Calculate the MSE between 'simple' and 'simple_noise' (original noise level)
mse_noise = mean((simple_noise - simple).^2);

% Calculate the MSE between 'simple' and the filtered version of 'simple_noise'
mse_filtered_noise = mean((simple - yy3).^2);

% Calculate distortion due to noise in the output
mse_distortion = mse_filtered_noise - mse_filter2;

disp(['MSE of noise: ', num2str(mse_noise)]);
disp(['MSE after filtering (signal recovery): ', num2str(mse_filtered_noise)]);
disp(['Distortion due to noise in output: ', num2str(mse_distortion)]);


%% Part 1c: Running Average Filters in 1-D
% Define 3-point, 5-point, and 9-point running average filters
bb3 = ones(3, 1) / 3;
bb5 = ones(5, 1) / 5;
bb9 = ones(9, 1) / 9;

% Apply each filter to 'simple_noise'
yy4 = filter2(bb3, simple_noise, 'same');
yy5 = filter2(bb5, simple_noise, 'same');
yy6 = filter2(bb9, simple_noise, 'same');

% Plot the original, filtered signals, and filter coefficients
figure;
subplot(4, 2, 1); plot(simple); title('Original Signal (simple)');
subplot(4, 2, 2); stem(bb3); title('3-point Filter Coefficients');

subplot(4, 2, 3); plot(yy4); title('Filtered Signal (3-point)');
subplot(4, 2, 4); stem(bb5); title('5-point Filter Coefficients');

subplot(4, 2, 5); plot(yy5); title('Filtered Signal (5-point)');
subplot(4, 2, 6); stem(bb9); title('9-point Filter Coefficients');

subplot(4, 2, 7); plot(yy6); title('Filtered Signal (9-point)');

% Compute MSE for each filtered signal
mse_3point = mean((simple - yy4).^2);
mse_5point = mean((simple - yy5).^2);
mse_9point = mean((simple - yy6).^2);

disp(['MSE for 3-point filter: ', num2str(mse_3point)]);
```

```
disp(['MSE for 5-point filter: ', num2str(mse_5point)]);
disp(['MSE for 9-point filter: ', num2str(mse_9point)]);


%% Part 1d: Tapered Smoothing Filter in 1-D
% Define widths for tapered smoothing filter
widths = [0.5, 0.75, 1.0];
yy7 = cell(1, 3);
mse_tapered = zeros(1, 3);

% Generate and apply filters with different widths
figure;
for i = 1:3
    bb_tapered = g_smooth(widths(i));
    yy7{i} = filter2(bb_tapered, simple_noise, 'same');

    % Plot filtered signals and coefficients
    subplot(3, 2, 2*i - 1);
    plot(yy7{i});
    title(['Filtered Signal (Width = ', num2str(widths(i)), ')']);

    subplot(3, 2, 2*i);
    stem(bb_tapered);
    title(['Tapered Filter Coefficients (Width = ', num2str(widths(i)), ')']);

    % Compute MSE for each filtered signal
    mse_tapered(i) = mean((simple - yy7{i}).^2);
end

disp(['MSE for tapered filter (width 0.5): ', num2str(mse_tapered(1))]);
disp(['MSE for tapered filter (width 0.75): ', num2str(mse_tapered(2))]);
disp(['MSE for tapered filter (width 1.0): ', num2str(mse_tapered(3))]);
```



>> FR

MSE using filter: 160.325

MSE using filter2: 43.6464

MSE of noise: 19.7631

MSE after filtering (signal recovery): 48.3135

Distortion due to noise in output: 4.6671

MSE for 3-point filter: 24.5223

MSE for 5-point filter: 36.2779

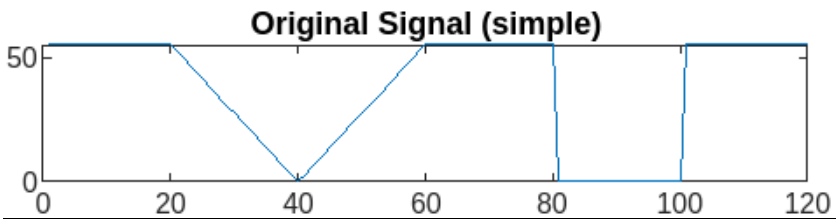
MSE for 9-point filter: 60.9562

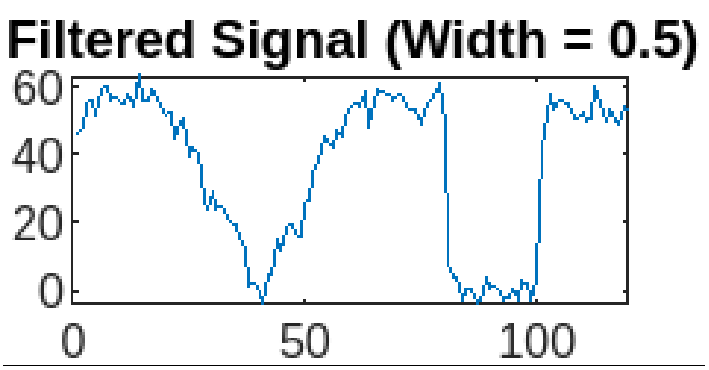
MSE for tapered filter (width 0.5): 14.9318

MSE for tapered filter (width 0.75): 16.9504

MSE for tapered filter (width 1.0): 21.4572

>>





Output for Part 1d:

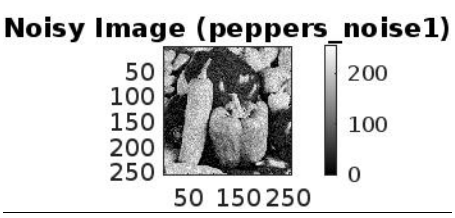
Compare the best tapered filter MSE with the best moving average filter MSE and original noise.

Part 2.

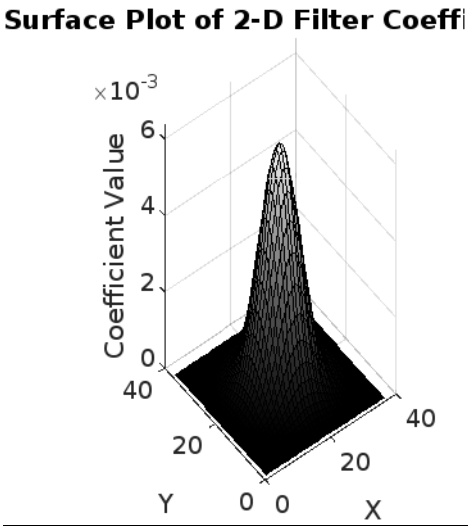
```
% Part 2: Noise Reduction on Images

% Load the images
peppers = imread('peppers.tif');  % Noise-free image
peppers_noise1 = imread('peppers_noise1.tif');  % Image corrupted with random noise

%% Part 2a: Display Original and Noisy Images Side-by-Side

% Display the original and noisy images
figure;
subplot(1,2,1);
imagesc(peppers);
colormap gray;
axis image;
colorbar;
title('Original Image (peppers)');

subplot(1,2,2);
imagesc(peppers_noise1);
colormap gray;
axis image;
colorbar;
title('Noisy Image (peppers\_noise1)');

% Calculate and display the Mean-Squared Error (MSE) between original and noisy image
mse_peppers_noise = mean((double(peppers(:)) - double(peppers_noise1(:))).^2);
disp(['MSE between original and noisy image: ', num2str(mse_peppers_noise)]);


%% Part 2b: Visualize 2-D Filter Coefficients (Width = 5)

% Generate 2-D filter coefficients with g_smooth2 (Width = 5)
bb2d = g_smooth2(5);

% Display the coefficients as an image and as a surface plot
figure;
subplot(1,2,1);
imagesc(bb2d);
colormap gray;
colorbar;
title('2-D Filter Coefficients (g\_smooth2 width = 5)');

subplot(1,2,2);
surf(bb2d);
title('Surface Plot of 2-D Filter Coefficients');
xlabel('X');
ylabel('Y');
zlabel('Coefficient Value');
```

```
%% Part 2c: Minimizing MSE with Optimal Filter Width

% Range of filter widths to test
widths = 1:0.5:10;
mse_values = zeros(length(widths), 1);

% Calculate MSE for each filter width by applying g_smooth2
for i = 1:length(widths)
    bb2d = g_smooth2(widths(i));
    filtered_image = filter2(bb2d, double(peppers_noise1), 'same');
    mse_values(i) = mean((double(peppers(:)) - filtered_image(:)).^2);
end

% Find the minimum MSE and corresponding width
[min_mse, min_index] = min(mse_values);
optimal_width = widths(min_index);
disp(['Optimal width for g\_smooth2: ', num2str(optimal_width)]);
disp(['Minimum MSE with optimal width: ', num2str(min_mse)]);

% Plot MSE as a function of width
figure;
plot(widths, mse_values, '-o');
xlabel('Filter Width');
ylabel('Mean-Squared Error');
title('MSE vs Filter Width for g\_smooth2 Filter');



%% Part 2d: Display Filtered Image with Optimal Width

% Apply the optimal width filter to the noisy image
bb2d_optimal = g_smooth2(optimal_width);
filtered_image_optimal = filter2(bb2d_optimal, double(peppers_noise1), 'same');

% Display the filtered image with the smallest MSE
figure;
imagesc(filtered_image_optimal);
colormap gray;
axis image;
colorbar;
title(['Filtered Image (Optimal Width = ', num2str(optimal_width), ')']);


%% Part 2e: Visual Comparison of Filtered Images with Different Widths

% Try different widths to visually compare filtered images
widths_to_compare = [2, optimal_width, 8];
figure;
for i = 1:length(widths_to_compare)
    bb2d_compare = g_smooth2(widths_to_compare(i));
    filtered_image_compare = filter2(bb2d_compare, double(peppers_noise1), 'same');

    subplot(1, length(widths_to_compare), i);
```

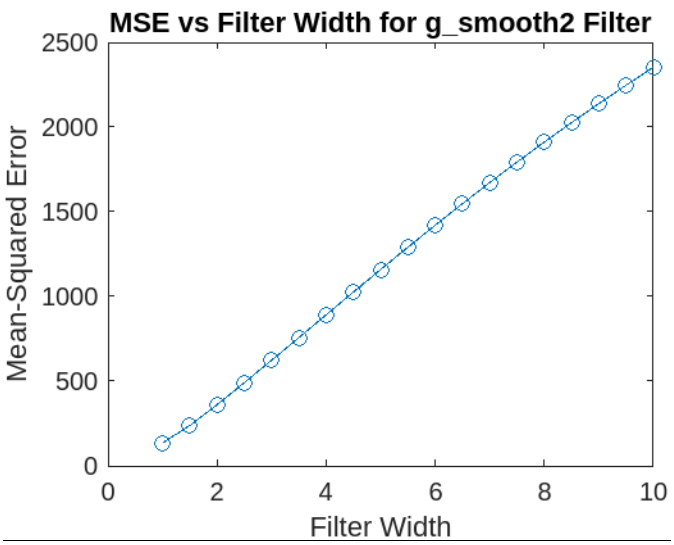
```
    imagesc(filtered_image_compare);
    colormap gray;
    axis image;
    colorbar;
    title(['Filtered Image (Width = ', num2str(widths_to_compare(i)), ')']);
end
```



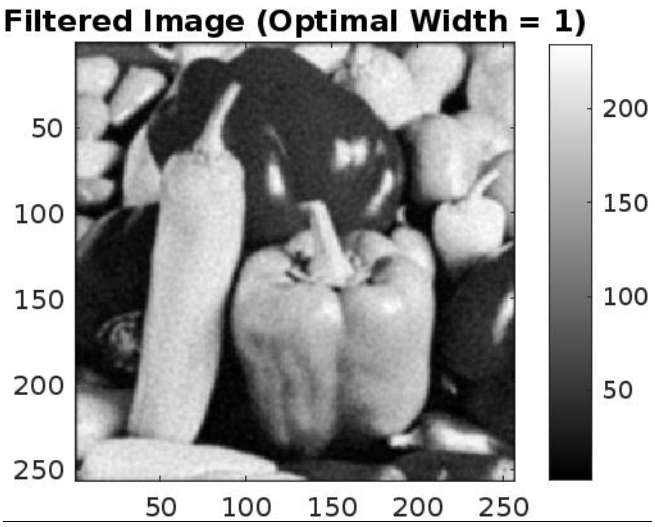
Outcome of Part 2a: Display and initial MSE
Observes the visual difference between the images and quantifies noise level with MSE.



Outcome of Part 2b:
Provides an understanding of the spatial distribution and intensity of filter coefficients.



Outcome of Part 2c:
Identifies the optimal width that minimizes the MSE, suggesting the best filter width for noise reduction.



Outcome of Part 2d:

The filtered image should appear smoother with reduced noise while preserving as much detail as possible

Outcome of Part 2e:

Allows visual inspection to see if any of the widths produce an image that looks better than the one with the minimum MSE.

>> NoiseImage

MSE between original and noisy image: 510.7131

Optimal width for g_smooth2: 1

Minimum MSE with optimal width: 135.48

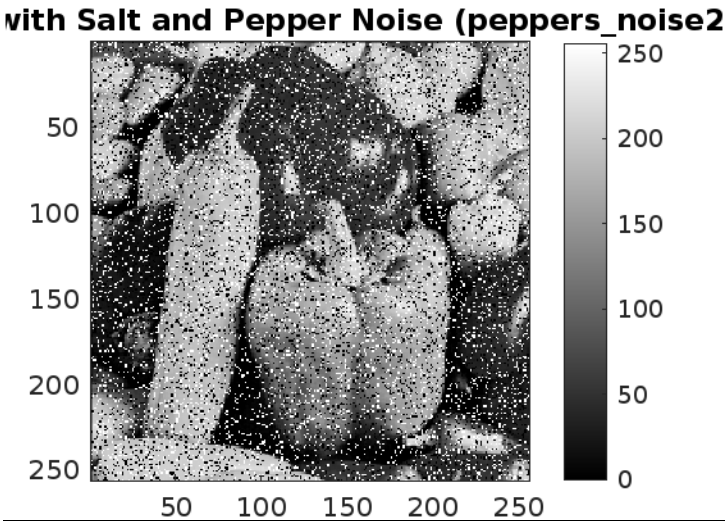
>>

Part 3.

```
% Part 3: Salt and Pepper Noise in Images

% Load the noise-corrupted image (salt and pepper noise)
peppers_noise2 = imread('peppers_noise2.tif');  % Image corrupted with salt and
pepper noise

%% Part 3a: Examining the Salt and Pepper Noise

% Display the noisy image
figure;
imagesc(peppers_noise2);
colormap gray;
axis image;
colorbar;
title('Image with Salt and Pepper Noise (peppers\_noise2)');

% Calculate and display the Mean-Squared Error (MSE) between noisy image and original
image
mse_salt_pepper = mean((double(peppers(:)) - double(peppers_noise2(:))).^2);
disp(['MSE between original and salt & pepper noisy image: ',
num2str(mse_salt_pepper)]);



%% Part 3b: Using Low-Pass Filter to Remove Salt and Pepper Noise

% Apply a g_smooth2 filter of width 1.3 (close to optimal for random noise)
bb2d_smooth = g_smooth2(1.3);
filtered_image_lowpass = filter2(bb2d_smooth, double(peppers_noise2), 'same');

% Display the low-pass filtered image
figure;
imagesc(filtered_image_lowpass);
colormap gray;
axis image;
colorbar;
title('Low-Pass Filtered Image (Width = 1.3)');

% Calculate and display MSE after low-pass filtering
mse_lowpass = mean((double(peppers(:)) - filtered_image_lowpass(:)).^2);
disp(['MSE after low-pass filtering: ', num2str(mse_lowpass)]);



%% Part 3c: Using Median Filters to Remove Salt and Pepper Noise
```

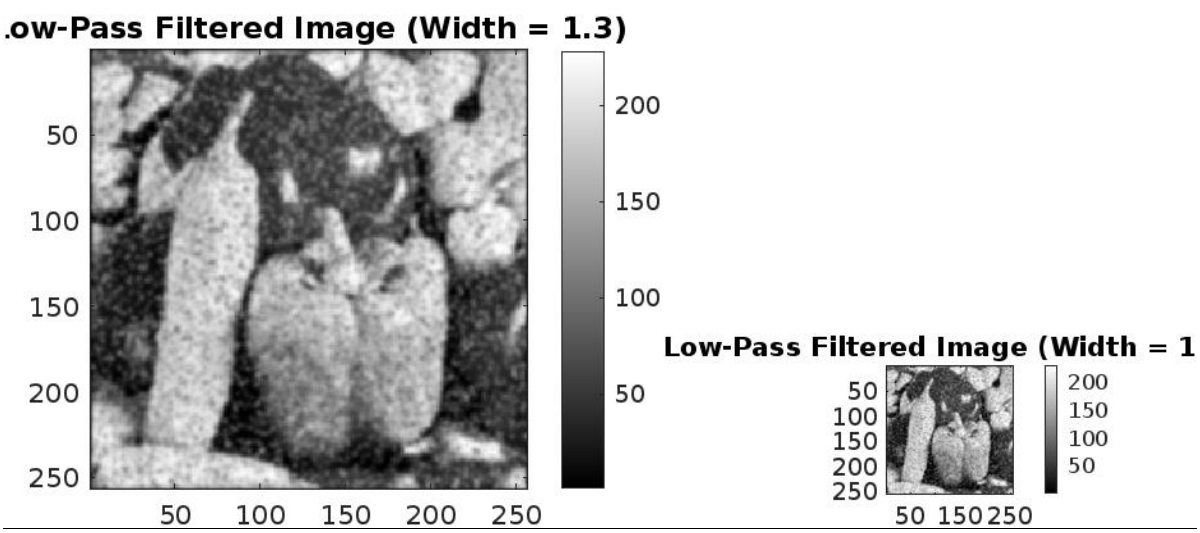
```
% Apply median filters with different orders (3 and 5)
filtered_image_median3 = medfilt2(peppers_noise2, [3 3]);
filtered_image_median5 = medfilt2(peppers_noise2, [5 5]);

% Display the median-filtered images side-by-side
figure;
subplot(1,2,1);
imagesc(filtered_image_median3);
colormap gray;
axis image;
colorbar;
title('Median Filtered Image (Order = 3)');

subplot(1,2,2);
imagesc(filtered_image_median5);
colormap gray;
axis image;
colorbar;
title('Median Filtered Image (Order = 5)');

% Calculate and display MSE for both median filters
mse_median3 = mean((double(peppers(:)) - double(filtered_image_median3(:))).^2);
mse_median5 = mean((double(peppers(:)) - double(filtered_image_median5(:))).^2);
disp(['MSE after median filtering (Order = 3): ', num2str(mse_median3)]);
disp(['MSE after median filtering (Order = 5): ', num2str(mse_median5)]);



%% Part 3d: Visual Comparison and Analysis

% Display the original and all filtered images for visual comparison
figure;
subplot(2,2,1);
imagesc(peppers_noise2);
colormap gray;
axis image;
colorbar;
title('Salt and Pepper Noisy Image');

subplot(2,2,2);
imagesc(filtered_image_lowpass);
colormap gray;
axis image;
colorbar;
title('Low-Pass Filtered Image (Width = 1.3)');

subplot(2,2,3);
imagesc(filtered_image_median3);
colormap gray;
axis image;
colorbar;
title('Median Filtered Image (Order = 3)');

subplot(2,2,4);
imagesc(filtered_image_median5);
```

```
colormap gray;
axis image;
colorbar;
title('Median Filtered Image (Order = 5)');
```



Outcome of Part 3a:

Displays the image with salt and pepper noise and provides the MSE, giving a baseline for how much distortion is present.



Outcome of Part 3b:

Shows the effect of a low-pass filter on salt and pepper noise and calculates the MSE for comparison with other filters.

Outcome of Part 3c:

Compares the results of different median filters, calculating MSE and observing how well each filter removes salt and pepper noise.

>> salt

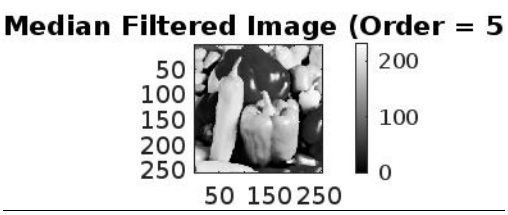
MSE between original and salt & pepper noisy image: 4394.6881

MSE after low-pass filtering: 641.7641

MSE after median filtering (Order = 3): 158.9117

MSE after median filtering (Order = 5): 203.3004

>>



Outcome of Part 3d:

Allows for visual inspection of the different filtering approaches. Observations will focus on how each filter handles noise

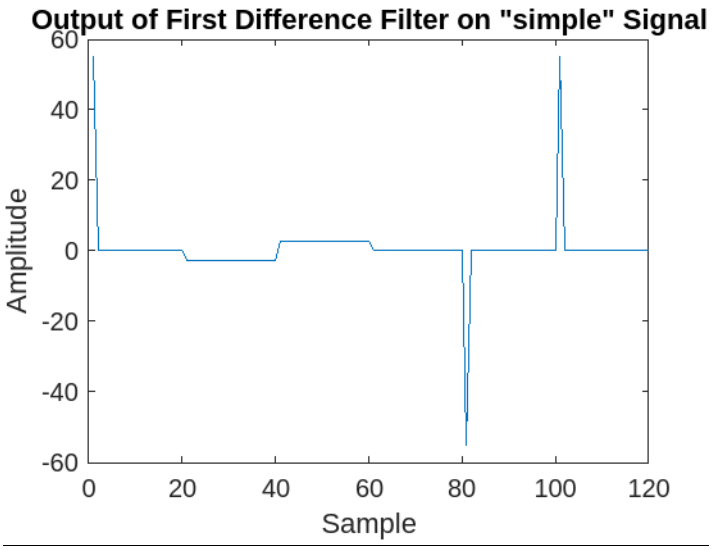
while preserving image details. Median filters typically perform better with salt and pepper noise compared to low-pass filters.

<u>Part 4.</u>

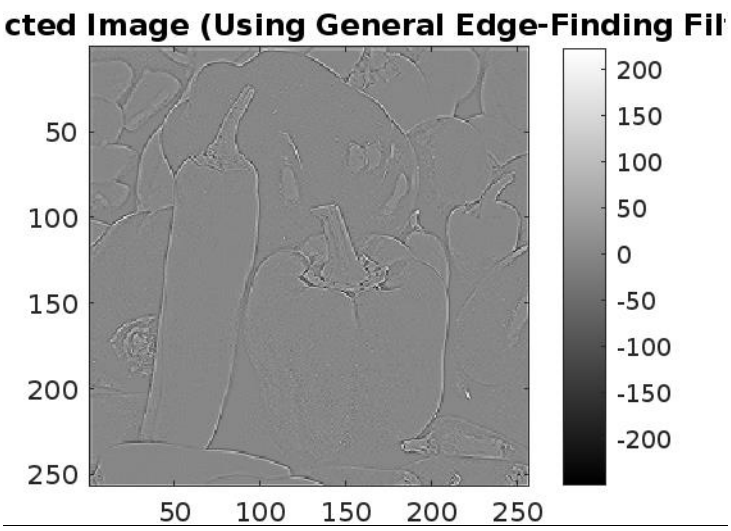
```
% Part 4: Edge-Finding and Enhancing

% Load the original 'simple' signal and 'peppers' image
load('lab2_data.mat');  % Contains 'simple' signal for Part 4a
peppers = imread('peppers.tif');  % Noise-free image for Part 4b onwards
```

```
%% Part 4a: Applying a First Difference Filter to the 'simple' Signal

% Define a first difference filter
bb_diff = [1; -1];

% Apply the filter using the filter command
yy_diff = filter(bb_diff, 1, simple);

% Plot the resulting signal after applying the first difference filter
figure;
plot(yy_diff);
title('Output of First Difference Filter on "simple" Signal');
xlabel('Sample');
ylabel('Amplitude');


%% Part 4b: Applying a General Edge-Finding Filter to an Image

% Define a 2D edge-finding filter
bb_edge = [0.25, -1, 0.25; -1, 3, -1; 0.25, -1, 0.25];

% Apply the edge-finding filter to the 'peppers' image using filter2
filtered_image_edge = filter2(bb_edge, double(peppers), 'same');

% Display the edge-detected image
figure;
imagesc(filtered_image_edge);
colormap gray;
axis image;
colorbar;
title('Edge-Detected Image (Using General Edge-Finding Filter)');



%% Part 4c: Sharpening the Image

% Define sharpening strengths to try
strengths = [0.5, 1, 1.5];

% Display the original and sharpened images side-by-side for comparison
figure;
subplot(2,2,1);
imagesc(peppers);
colormap gray;
axis image;
colorbar;
title('Original Image (peppers)');

for i = 1:length(strengths)
    % Use sharpen function to apply sharpening
    sharpened_image = sharpen(double(peppers), strengths(i));

    % Display the sharpened image
    subplot(2,2,i+1);
```

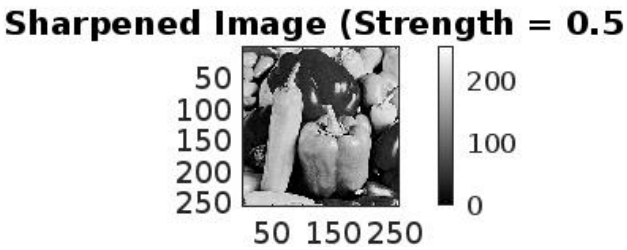
```
    imagesc(sharpened_image);
    colormap gray;
    axis image;
    colorbar;
    title(['Sharpened Image (Strength = ', num2str(strengths(i)), ')']);
end



%% Part 4d: Using Sharpening to "De-blur" an Image

% Load the blurred image
peppers_blur = imread('peppers_blur.tif');  % Blurred version of 'peppers'

% Calculate the RMS error between the blurred and original image as a baseline
rms_blur = sqrt(mean((double(peppers(:)) - double(peppers_blur(:))).^2));
disp(['RMS error between original and blurred image: ', num2str(rms_blur)]);

% Apply sharpening with various strengths to find the minimum RMS error
strengths = 0.5:0.1:2.0;
rms_errors = zeros(length(strengths), 1);

for i = 1:length(strengths)
    sharpened_image = sharpen(double(peppers_blur), strengths(i));
    rms_errors(i) = sqrt(mean((double(peppers(:)) - sharpened_image(:)).^2));
end

% Find the strength that gives the minimum RMS error
[min_rms, min_index] = min(rms_errors);
optimal_strength = strengths(min_index);
disp(['Optimal sharpening strength for de-blurring: ', num2str(optimal_strength)]);
disp(['Minimum RMS error with optimal strength: ', num2str(min_rms)]);

% Display the "de-blurred" image using the optimal sharpening strength
figure;
subplot(1,2,1);
imagesc(peppers_blur);
colormap gray;
axis image;
colorbar;
title('Blurred Image');

subplot(1,2,2);
sharpened_optimal = sharpen(double(peppers_blur), optimal_strength);
imagesc(sharpened_optimal);
colormap gray;
axis image;
colorbar;
title(['"De-blurred" Image (Optimal Strength = ', num2str(optimal_strength), ')']);
```



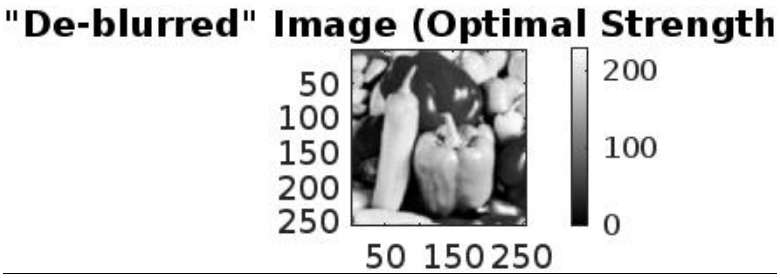
Outcome of Part 4a:
The plot should display five distinct non-zero "features" corresponding to changes in 'simple'.
These features typically represent edges or transitions in the signal.



Outcome of Part 4b:
The resulting image should highlight the edges in the 'peppers' image, with high values along edges.
Zooming in shows that edges are not simple lines but rather regions with intensity changes.



Outcome of Part 4c:
As the strength parameter increases, edges and details become more pronounced. However, higher sharpening levels can introduce noise or graininess, especially along edges.



Outcome of Part 4d:

The optimal sharpening strength reduces RMS error, partially restoring the blurred image.

Visual comparison shows that edges are sharper, though very high strengths may over-enhance noise

>> edge

RMS error between original and blurred image: 18.4912

Optimal sharpening strength for de-blurring: 2

Minimum RMS error with optimal strength: 15.0966

>>